



Базовые принципы распределенного обучения нейросетей. Parameter Server.

Large Scale ML: Лекция 6, Машинное обучение на
больших данных

Ведущий лекции:

Владимир Суловьев

Математик-разработчик в группе Внутренних ML
Продуктов

18:30 ✓✓



О чем будем говорить?

Введение и мотивация

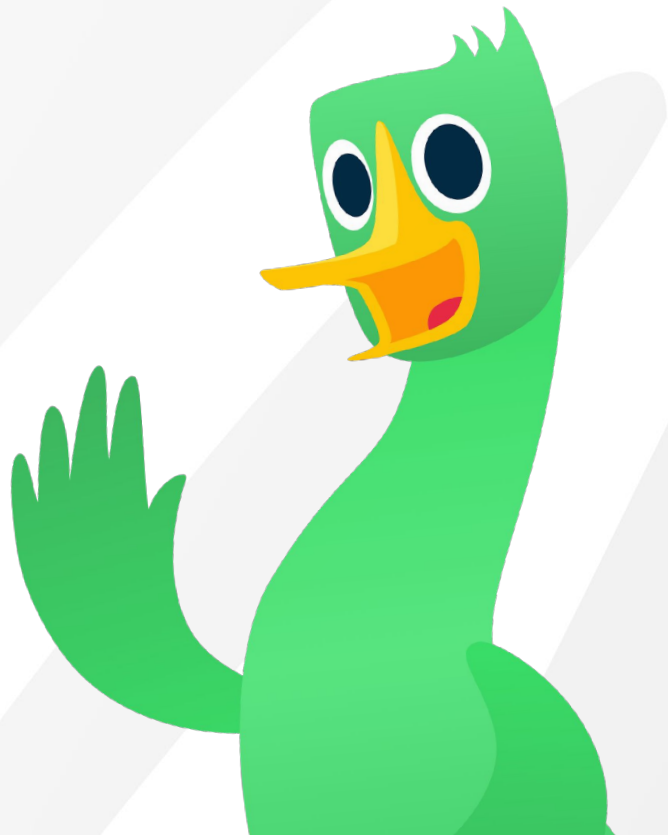
Математика Data Parallelism

Архитектура Parameter Server

Стратегии синхронизации

Ограничения PS и переход к современности

Федеративное обучение



01



Введение и мотивация

Зачем нам масштабироваться?

Закон масштабирования (Scaling Laws):

Исследования (в частности, от OpenAI: Kaplan et al., "Scaling Laws for Neural Language Models") доказали эмпирический закон: качество модели предсказуемо растет при одновременном увеличении трех факторов:

- Объем вычислений (Compute / FLOPs).
- Размер датасета.
- Количество параметров модели.

Значит, чтобы получать SOTA результаты, мы обязаны увеличивать модели и объемы данных.

Челленджи обучения

Две главные "Стены" (The Two Walls), в которые мы упираемся:

- **Time / Compute Wall:**

Даже если модель влезает в одну самую мощную видеокарту (например, H100), обучение на терабайтах данных займет годы.

(Обучение ResNet-50 на ImageNet на 1 GPU занимает несколько дней. Обучение GPT-3 на 1 GPU заняло бы ~355 лет.)

Решение: Нам нужно больше процессоров/GPU, чтобы обрабатывать данные параллельно.

- **Memory Wall:**

Аппаратное ограничение. Топовые GPU сегодня имеют 80-144 ГБ памяти (VRAM).

Решение: Нам нужно дробить саму модель и раскидывать её по разным устройствам.

Основные парадигмы распределенного обучения

Data Parallelism (DP) — Параллелизм по данным:

Суть: У нас есть "бесконечные" данные, но модель небольшая (влезает в одну GPU).

Как работает:

1. Мы копируем целиком одну и ту же модель на N карточек (воркеров).
2. Мы берем большой батч данных, рубим его на N частей (shards).
3. Каждая видеокарта делает forward/backward pass только на своей части данных.
4. Затем они обмениваются градиентами, чтобы обновить веса синхронно.

Model Parallelism (MP) — Параллелизм по модели:

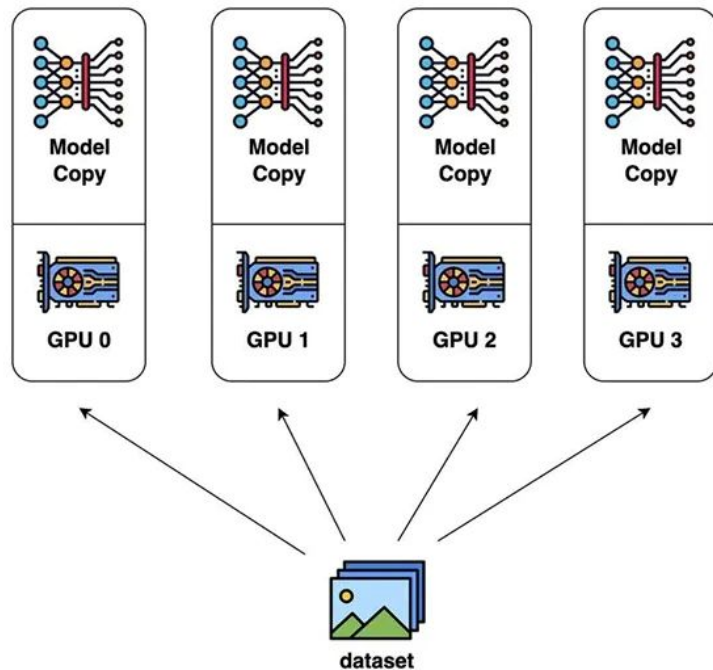
Суть: Модель огромная, она не влезает в одну GPU (Memory Wall).

- Pipeline Parallelism (Конвейерный). Режем модель по слоям. Данные текут последовательно от одной карты к другой.
- Tensor Parallelism (Тензорный). Режем математику внутри одного слоя.

Синхронизация

В Data Parallelism у нас на старте есть N идентичных копий модели на N машинах. Каждая машина пропустила через себя разные данные и получила разные градиенты (направления, куда нужно сдвинуть веса).

Главный вызов Data Parallelism: Нам нужен надежный, быстрый и масштабируемый механизм коммуникации. Нам нужно собрать все градиенты, усреднить их и раздать новые, одинаковые веса всем устройствам.



02



Математика Data Parallelism



Вспомним классический SGD

На одной машине (Single Node) шаг обучения выглядит так:

1. Выбираем мини-батч данных B .
2. Считаем функцию потерь $L(w)$ на этом батче.
3. Находим градиент: $g = \nabla_w L(w, B)$.
4. Обновляем веса: $w_{t+1} = w_t - \eta \cdot g$,
где η — learning rate.

Переход к распределенному SGD (D-SGD)

1. Разделение данных: Мы берем огромный батч данных и делим его на K равных частей.
2. Каждая часть B_k отправляется на свой воркер k .
3. Локальный расчет: Каждый воркер независимо вычисляет градиент на своих данных:

$$g_k = \nabla_w L(w_t, B_k)$$

4. Важно: на момент расчета у всех воркеров должны быть одинаковые веса w_t .
5. Агрегация (Магия усреднения): Чтобы модель училась так, будто она видит все данные сразу, нам нужно получить Глобальный градиент (G):

$$G = 1/K \sum g_k$$

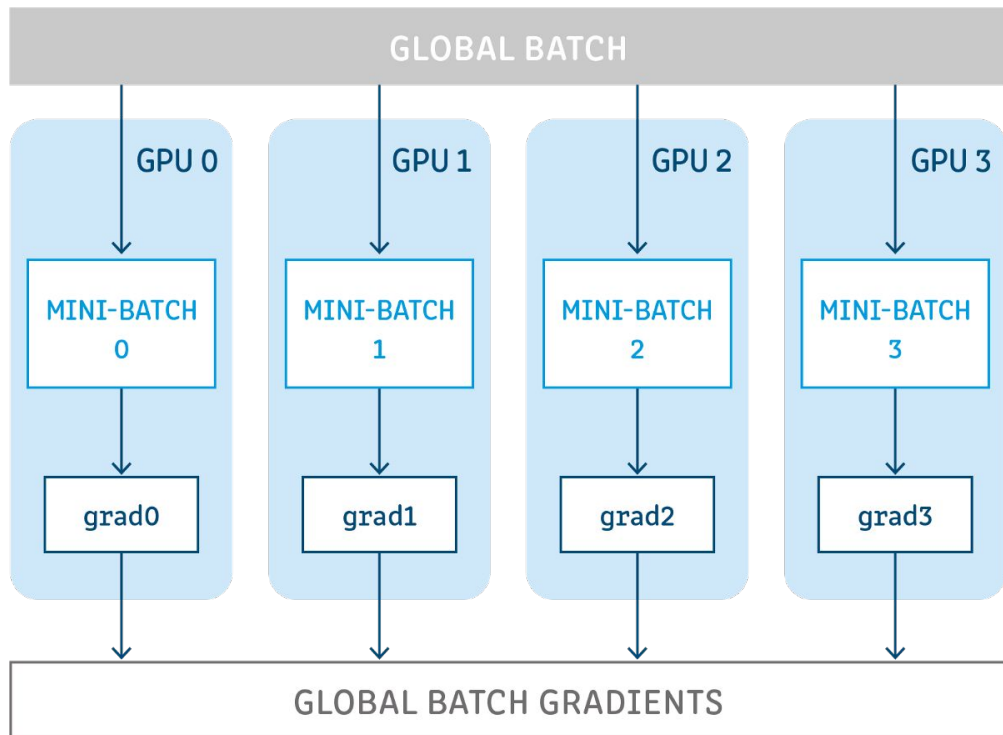
6. Обновление: Теперь мы можем обновить веса, используя этот усредненный градиент:

$$w_{t+1} = w_t - \eta \cdot G$$

Понятие "Global Batch Size" и его последствия

Если на одной GPU мы учили с батчем 32, а теперь у нас 128 GPU, наш "эффективный" батч стал $32 \times 128 = 4096$.

Большой батч делает градиент менее "шумным". С одной стороны, это хорошо. С другой — модель может начать "застрывать" в острых минимумах и хуже обобщаться (Generalization Gap).



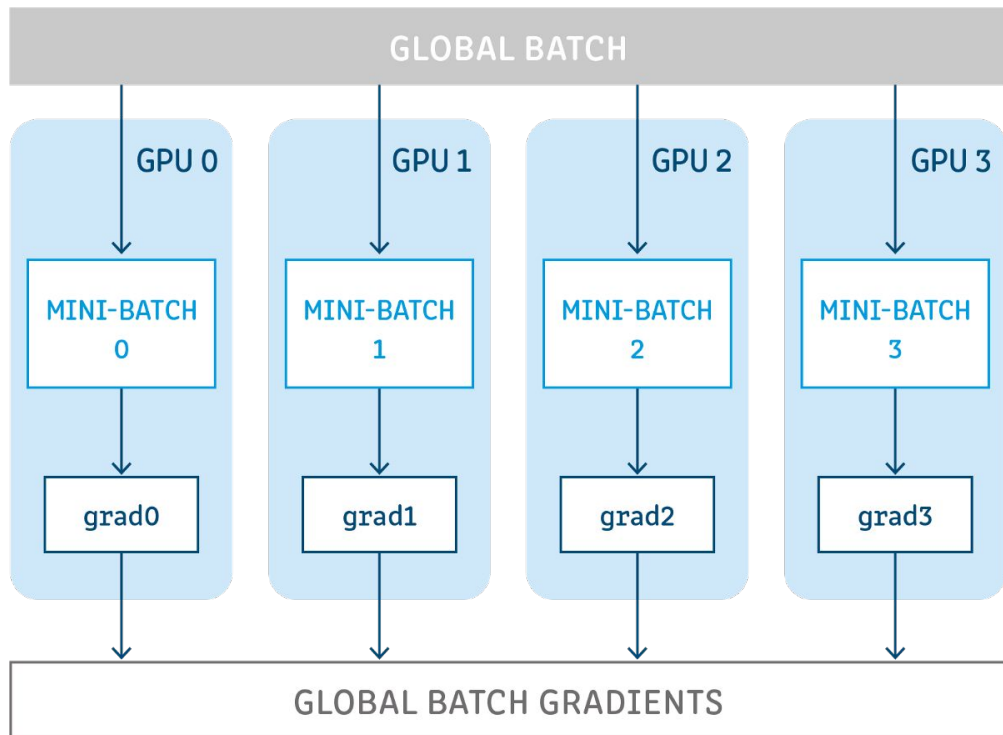
Понятие "Global Batch Size" и его последствия

Linear Scaling Rule (Правило линейного масштабирования):

Эмпирическое правило (Goyal et al., 2017):

При увеличении батча в K раз, нужно увеличить Learning Rate (η) тоже в K раз, чтобы сохранить динамику обучения.

Замечание: Это работает не бесконечно. При очень больших батчах точность всё равно начинает падать.



Итого: жизненный цикл итерации

1. Broadcast: Машины синхронизируют веса (у всех w_t должно быть идентичным).
2. Compute (Forward + Backward Pass): Каждая машина считает свои g_k . Это самая тяжелая часть для GPU (TFLOPS).
3. Communication (All-Reduce / Push-Pull): Машины пересылают градиенты друг другу или на сервер. Это самая тяжелая часть для сети (Gbps).
4. Apply: Обновление весов в памяти.

Главный конфликт распределенного ML:

Эффективность $\approx T / (T + C)$,

где T — время вычислений градиента, C — время пересылки градиентов по сети.

Т.е. если сеть медленная (C большое), то добавление новых GPU не ускорит обучение, они будут просто стоять и ждать сеть.



Вопросы?

03



Архитектура Parameter Server

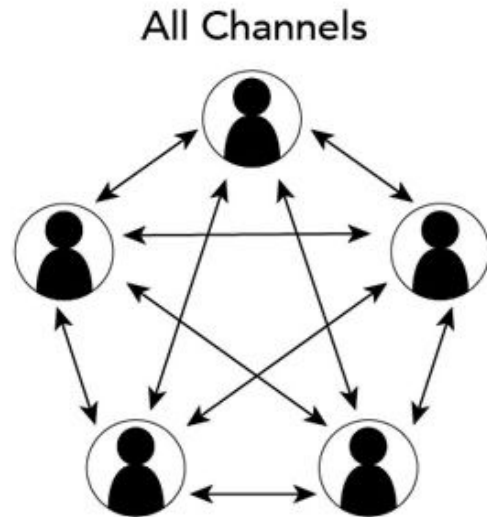
Проблема All-to-All

Мы выяснили, что N воркеров должны усреднить свои градиенты. Самый наивный подход: каждый воркер отправляет свои градиенты всем остальным воркерам.

Это топология All-to-All (Каждый с каждым).

Сложность сети: $O(N^2)$ соединений. Если у нас 1000 GPU, сеть просто рухнет от количества одновременных подключений и коллизий.

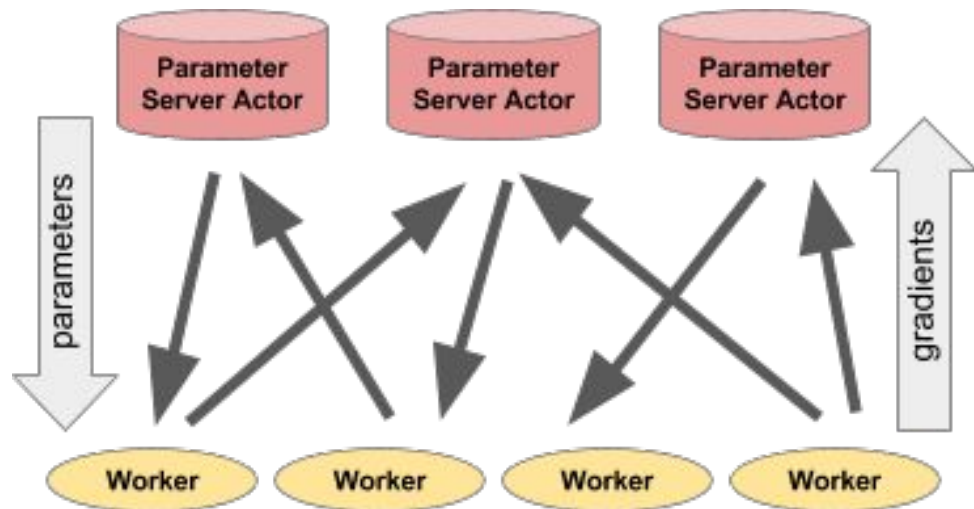
Решение (Концепция Parameter Server): Нам нужен координатор. В 2012 году Google (DistBelief), а затем в 2014 году Mu Li (создатель MXNet) формализовали архитектуру Parameter Server.



Концепция PS

Кластер логически (а часто и физически) делится на две группы узлов:

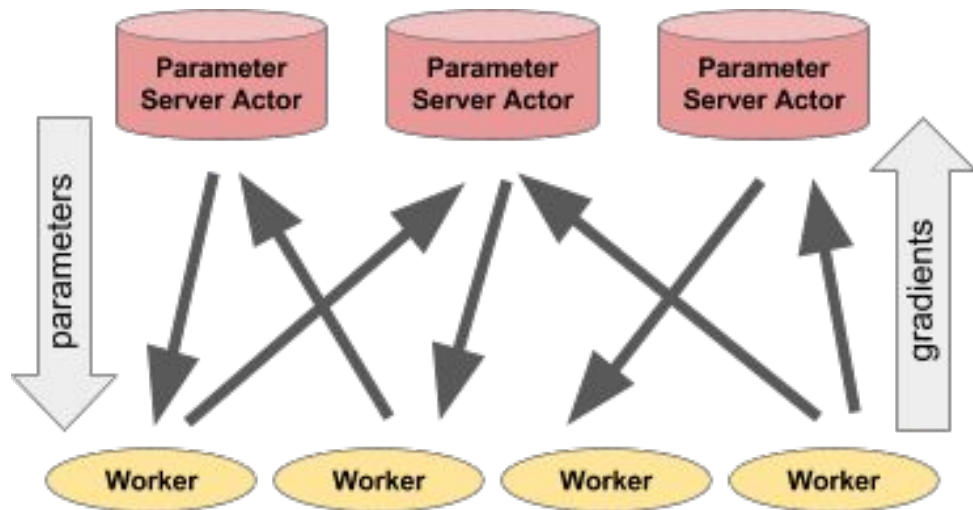
- **Workers:** Это узлы с GPU или TPU. Их задача — "молотить" данные. Они читают батч, делают forward/backward pass и получают градиенты. Они не хранят глобальное состояние модели.
- **Servers:** Это узлы (обычно мощные CPU с огромным объемом оперативной памяти). Их задача — хранить "золотую" (актуальную) копию весов модели, принимать градиенты от воркеров, усреднять их и делать шаг оптимизатора (например, применять Adam).



Концепция PS

Интерфейс общения (Push и Pull API):

- Push: Воркер отправляет вычисленные градиенты на сервер.
- Pull: Воркер запрашивает с сервера свежие обновленные веса перед началом следующего шага.



Абстракция: Распределенный KV-Store

Ключ (K): Идентификатор параметра. Например, `layer_1_weight_matrix` или `embedding_user_10452`.

Значение (V): Сам тензор для этого ключа.

Зачем нужна такая абстракция?

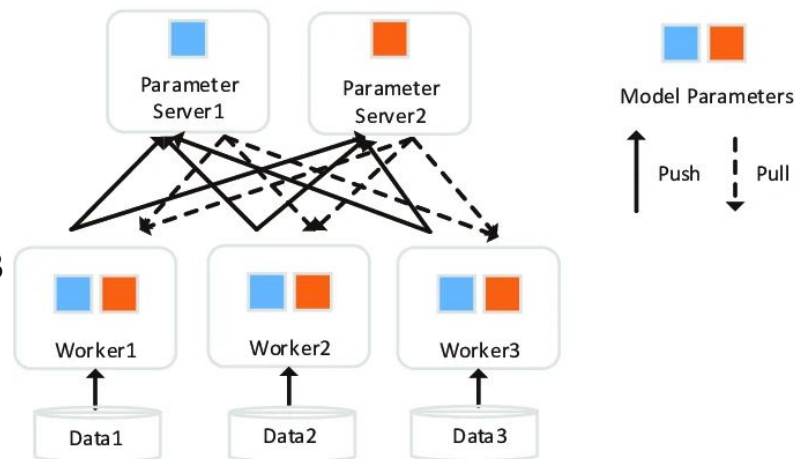
Для плотных моделей (CV, NLP): Воркеру всегда нужна вся модель целиком. Он делает `Pull(All_Keys)`.

Для разреженных моделей (Рекомендательные системы, Реклама): Представьте модель предсказания клика (CTR), где есть таблица эмбеддингов на 1 миллиард пользователей. Модель весит терабайты. Но в одном батче у воркера всего 1024 пользователя.

Магия PS: Воркер делает `Pull([Key_User_10, Key_User_55...])` — запрашивает только те 1024 вектора, которые нужны ему для текущего батча! После расчета градиентов он делает `Push` только для этих же 1024 ключей. Это экономит 99.9% сетевого трафика.

Шардирование и балансировка нагрузки

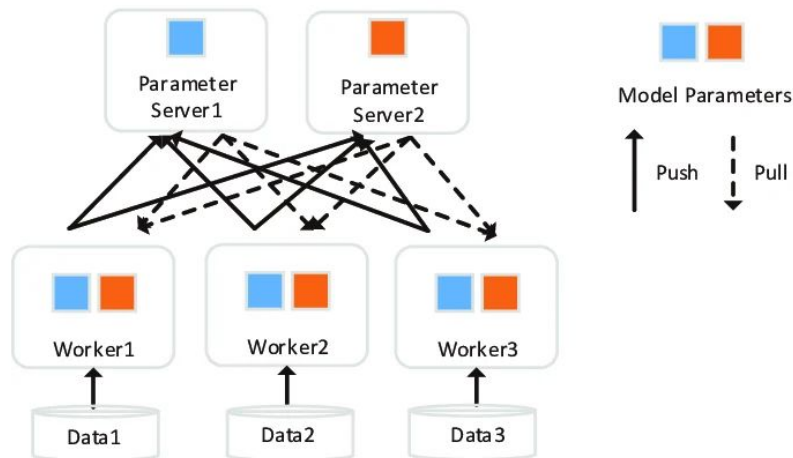
- Server Group: Мы используем не один сервер параметров, а целый кластер серверов (например, 10 машин).
- Шардирование (Sharding): Глобальный KV-Store "нарезается" на куски. Каждый сервер параметров хранит только свою часть ключей.
- Проблема "Горячих ключей" (Hot Keys Bottleneck): В рекомендательных системах данные распределены по закону Ципфа (Zipf's law). Есть очень популярные товары (например, "iPhone 15") или слова (предлоги "и", "в").



Сетевая топология

Двудольный граф:

- У нас есть множество Воркеров W и множество серверов S .
- Каждый воркер имеет TCP-соединение с каждым сервером. (Матрица $W \times S$ соединений).
- Пусть размер модели (сумма всех тензоров) = M мегабайт.
- За один шаг каждый воркер скачивает M (Pull) и отправляет M (Push). Итого: $2M$ трафика на воркер.
- Если серверов мало, а воркеров много (что типично: GPU дорогие, их покупают много, а CPU-серверов выделяют меньше), то на сетевую карту одного сервера падает огромная нагрузка.



Производительность обучения с использованием **Parameter Server** чаще всего упирается не в скорость GPU, а в пропускную способность сетевой карты серверов и коммутаторов.



Вопросы?

04



Стратегии синхронизации



Проблема рассинхронизации

Идеальный мир:

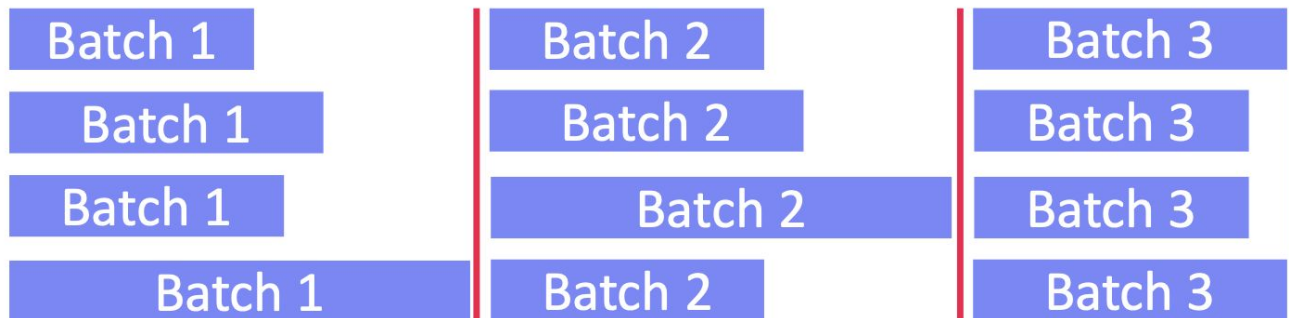
У нас есть 100 абсолютно одинаковых GPU, идеальная сеть без задержек, и каждый батч обрабатывается ровно за 1.000 секунды.

Реальный мир дата-центров:

1. Гетерогенное железо: В кластере могут быть старые GPU (T4) и новые (H100).
2. Сетевой джиттер: Пакеты данных могут задерживаться на коммутаторах.
3. Соседи по серверу: Кто-то запустил тяжелый процесс на CPU того же узла, где работает ваша GPU.
4. Разные данные: Обработка батча с длинными текстами занимает больше времени, чем с короткими (даже с паддингом).

Bulk Synchronous Parallel (BSP) — Синхронный режим

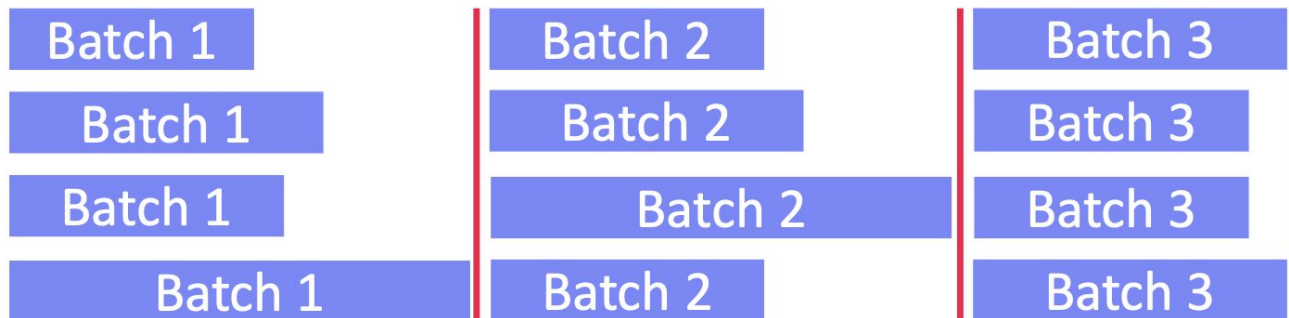
Никто не делает следующий шаг, пока все не закончат текущий.



1. Все воркеры скачивают веса w_t .
2. Все считают свои локальные градиенты g_i .
3. Сервер ждет, пока каждый из N воркеров пришлет свой Push.
4. Сервер усредняет все градиенты, делает шаг оптимизатора w_{t+1} и рассылает сигнал (или разрешает Pull), что можно начинать новую итерацию.

Bulk Synchronous Parallel (BSP) — Синхронный режим

Никто не делает следующий шаг, пока все не закончат текущий.

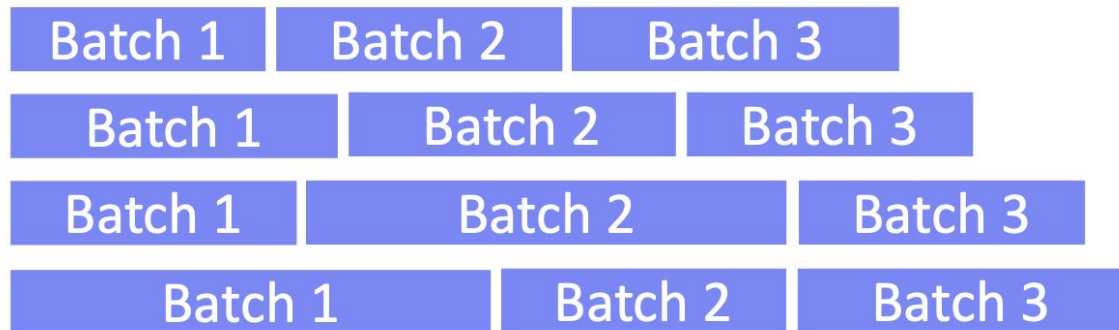


Плюсы: Это математически эквивалентно запуску обучения на одной огромной видеокарте с гигантским размером батча.

Минусы: Если 99 видеокарт закончили работу, а 1 "зависла" из-за сети на 10 секунд, то 99 дорогущих GPU просто простаивают (Idle state). Утилизация кластера падает на дно. Время обучения растягивается.

Asynchronous Parallel (ASP) — Асинхронный режим

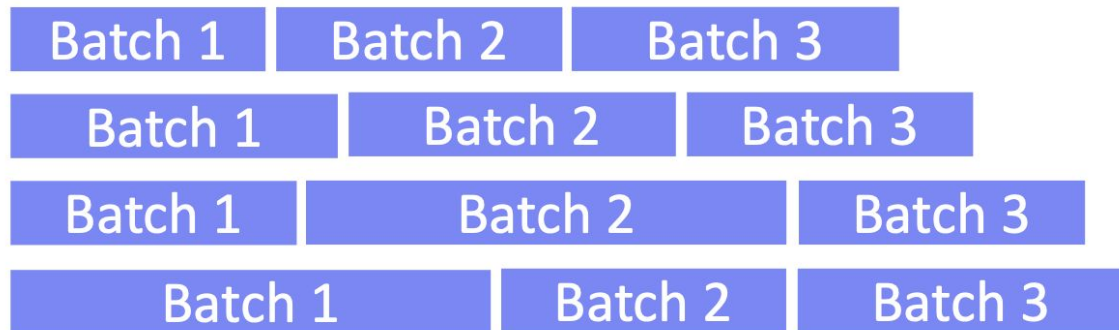
Полная анархия. Каждый работает в своем темпе, никого не ждем.



1. Воркер скачивает текущие веса (например, w_{100}).
2. Считает градиенты на своих данных.
3. Отправляет Push на сервер.
4. Сервер немедленно применяет этот градиент к весам и обновляет их (до w_{101}).

Asynchronous Parallel (ASP) — Асинхронный режим

Полная анархия. Каждый работает в своем темпе, никого не ждем.



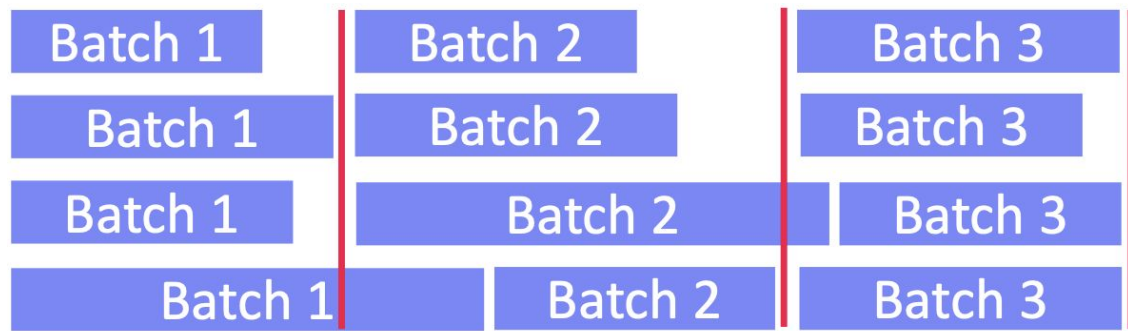
Плюсы: Утилизация оборудования 100%.

Минусы: Проблема устаревших градиентов.

Модель обучается быстро в секундах, но на графике loss'а получается ад. Модель может сходиться в 3 раза дольше по количеству эпох, или алгоритм может вообще разойтись.

Stale Synchronous Parallel (SSP) — Компромисс

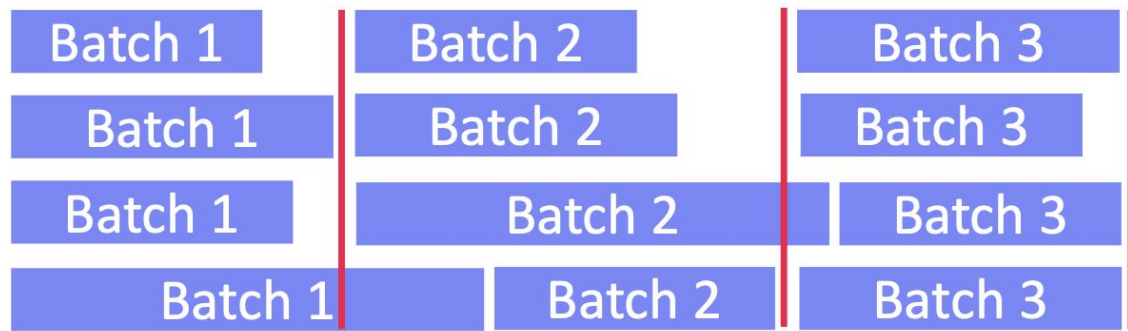
Ограниченная свобода. (Эта стратегия была предложена вместе с архитектурой Parameter Server (Mu Li, 2014), чтобы взять лучшее от обоих миров).



Мы вводим параметр **S** (Staleness Threshold — порог устаревания). Например, $S=3$. Самый быстрый воркер не имеет права обогнать самого медленного более чем на **S** шагов. Если Быстрый Воркер находится на шаге 10, а Медленный всё еще на шаге 6 (разница 4, что больше 3), то Сервер блокирует Быстрому Воркеру возможность скачать новые веса. Быстрый Воркер вынужден подождать, пока Медленный не закончит шаг 7.

Stale Synchronous Parallel (SSP) — Компромисс

Ограниченная свобода. (Эта стратегия была предложена вместе с архитектурой Parameter Server (Mu Li, 2014), чтобы взять лучшее от обоих миров).



- Это нивелирует проблему мелких сетевых задержек (джиттера). Если кто-то "запнулся" на долю секунды, кластер этого не заметит (как в ASP).
- Но при этом математики доказали, что если ограничить рассинхронизацию константой S , то алгоритм гарантированно сойдется, так как ошибка от устаревших градиентов имеет строгий математический предел.

Что выбрать?

В современных реалиях:

- Для плотных глубоких нейросетей (Transformer, CNN, LLM) устаревшие градиенты фатальны. Они слишком чувствительны к шуму. Поэтому индустрия вернулась к строгому BSP (Синхронному режиму). Проблему Stragglers решают покупкой дорогого однородного железа.
- Для разреженных рекомендательных систем (где обновляются гигантские таблицы эмбедингов, и разные воркеры почти не пересекаются по одним и тем же пользователям/айдишникам), вероятность конфликта мала. Там до сих пор активно используют ASP или SSP.



Вопросы?



Перерыв

05



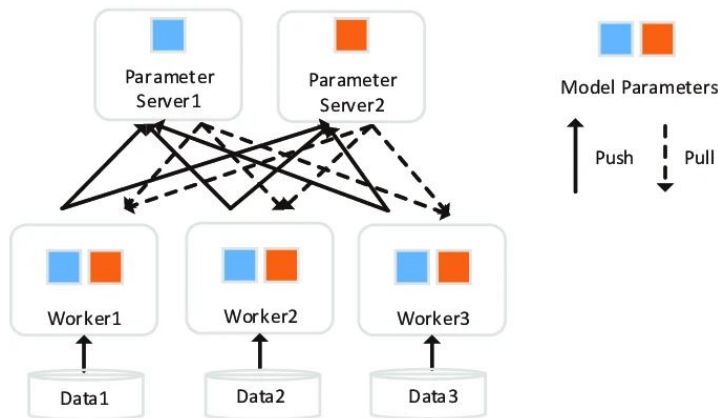
Ограничения PS и переход к современности

Главная проблема Parameter Server: Бутылочное горлышко

В PS архитектуре (двудольный граф) все N воркеров (Worker) стучатся на M серверов параметров (Server).

Представьте, что мы обучаем ResNet-50 или BERT. В таких моделях во время Forward/Backward pass задействованы все веса. Это значит, что на каждом шаге каждый из N воркеров должен:

- Отправить весь свой вектор градиентов (например, 1 ГБ) на серверы.
- Скачать всю обновленную модель (еще 1 ГБ) с серверов.



Главная проблема Parameter Server: Бутылочное горлышко

В PS архитектуре (двудольный граф) все N воркеров (Worker) стучатся на M серверов параметров (Server).

Математика краха:

- Суммарный трафик, который обрушивается на серверную группу за один шаг:
 $N \times 1 \text{ ГБ (Push)} + N \times 1 \text{ ГБ (Pull)} = 2N \text{ ГБ}.$
- Если у нас 100 GPU-воркеров и всего 5 CPU-серверов параметров, то на один сервер падает $(2 \times 100 \text{ ГБ}) / 5 = 40 \text{ ГБ}$ трафика в секунду!
- Сетевая карта сервера на 100 Гбит/с (около 12.5 ГБ/с) просто захлебнется. Воркеры с супербыстрыми A100/H100 будут 90% времени стоять и ждать, пока сервер прожует сеть.

Вывод: PS идеально масштабируется по количеству данных (добавляем воркеры), но ужасно масштабируется по размеру плотной модели (сеть сервера становится узким местом).

Эволюция: All-Reduce

Давайте избавимся от серверов вообще. Пусть воркеры сами договорятся между собой. Вместо того чтобы все слали всё в центр (All-to-One), мы выстраиваем воркеры в логическое кольцо.

Как это работает (очень кратко, тизер к следующей лекции):

- Воркер 1 шлет кусок своего градиента Воркеру 2. Воркер 2 прибавляет свой кусок и шлет Воркеру 3. И так по кругу.
- За $2(N-1)$ шагов каждый воркер получает полную, усредненную сумму градиентов.

В чем магия кольца?

Пропускная способность постоянна. Объем данных, который проходит через сетевую карту каждого воркера, практически не зависит от количества воркеров в кольце! (Он равен примерно $2 \times \text{размер_модели}$).

Когда Parameter Server все еще работает?

Сценарий разреженной модели (Sparse Model):

Рекомендательные системы (YouTube, TikTok, Яндекс.Директ).

Модель состоит из гигантской таблицы эмбеддингов (сотни гигабайт или терабайты: ID пользователя -> вектор, ID товара -> вектор).

Плотная часть (MLP/CTR-предсказатель) очень маленькая (пару мегабайт).

Почему All-Reduce здесь умрет:

В All-Reduce воркеры обязаны гонять по кольцу весь градиент целиком (все терабайты!). Это убьет любую сеть.

Когда Parameter Server все еще работает?

Сценарий разреженной модели (Sparse Model):

Рекомендательные системы (YouTube, TikTok, Яндекс.Директ).

Модель состоит из гигантской таблицы эмбеддингов (сотни гигабайт или терабайты: ID пользователя -> вектор, ID товара -> вектор).

Плотная часть (MLP/CTR-предсказатель) очень маленькая (пару мегабайт).

А вот PS здесь идеален:

На каждом шаге конкретному воркеру нужны эмбеддинги только для тех 1000 пользователей, которые попали к нему в текущий батч.

Воркер делает Pull только для 1000 строк из таблицы (это килобайты трафика!). Сервер параметров легко отдает эти крохи.

Воркер считает градиент только для этих 1000 строк и делает Push (снова килобайты).

Современные гибриды: BytePS

Это современная эволюция PS.

Идея:

- В кластере с All-Reduce у нас мощные GPU, соединенные быстрой сетью (NVLink/RoCE), но при этом простаивают мощные CPU и их оперативная память на тех же машинах.
- BytePS использует свободные ресурсы CPU и RAM на узлах с GPU в качестве "серверов параметров", создавая гибридную топологию, которая бьет и классический PS, и чистый All-Reduce (NCCL) на определенных типах моделей (особенно с гетерогенными слоями).



Вопросы?

06



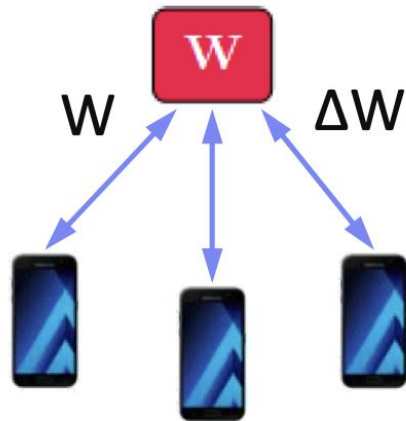
Федеративное
обучение



Обучение там, где лежат данные

Проблема традиционного ML: Чтобы обучить модель (например, T9 в клавиатуре смартфона или диагностику по снимкам МРТ), нам нужно собрать все данные в облако.

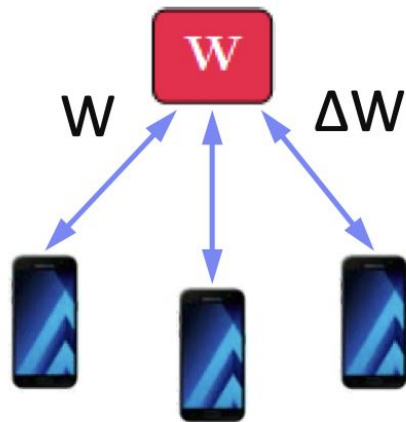
1. Приватность: Пользователи не хотят отдавать свои переписки, а больницы — истории болезней (GDPR, HIPAA).
2. Трафик: Передавать терабайты сырых данных с мобильных устройств дорого и долго.



Обучение там, где лежат данные

Федеративное обучение — это способ решения этой проблемы: мы не отправляем данные на сервер. Мы отправляем модель к данным.

- Данные остаются на устройствах (Edge devices: телефоны, IoT, локальные серверы больниц).
- На устройствах происходит локальное обучение, а на центральный сервер уходят только обновленные веса/градиенты.



Центральный сервер как Parameter Server

Центральный агрегатор (Server): Выступает в роли "глобального" Parameter Server. Он хранит Master Model.

Клиенты (Workers): Миллионы смартфонов или десятки клиник.

Цикл обучения:

1. Selection: Сервер выбирает подмножество активных клиентов (например, 100 из 1 000 000).
2. Broadcast: Сервер рассылает им текущие веса w_t .
3. Local Training: Каждый клиент делает несколько эпох обучения на своих личных данных.
4. Aggregation: Клиенты присылают обновленные веса. Сервер усредняет их.

FedAvg (Federated Averaging)

Почему не подходит обычный SGD?

В обычном распределенном ML мы обмениваемся градиентами после каждого мини-батча.

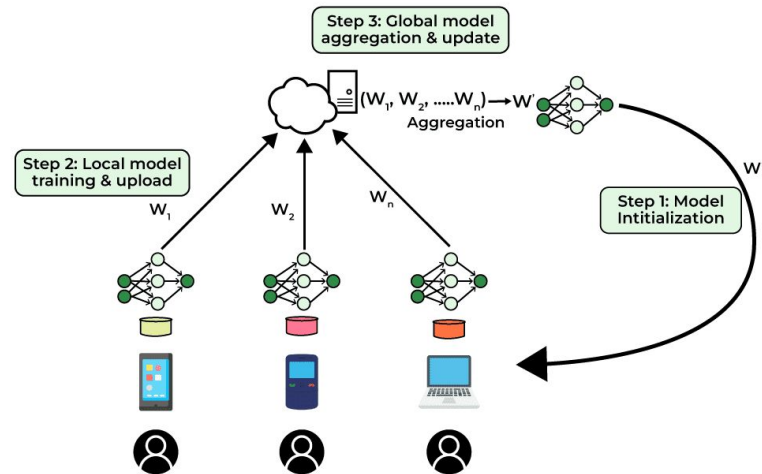
В FL сеть слишком медленная.

FedAvg (McMahan et al., 2017):

- Клиент делает не один шаг, а много эпох локально.
- Вместо градиентов передаются накопленные изменения весов Δw .

$$w_{t+1} = 1/n \sum_k n_k w_k,$$

где n_k — это количество данных на конкретном устройстве: аналог взвешенного среднего.



Три главных вызова Federated Learning

1. В дата-центре мы перемешиваем данные, а в FL данные распределены как попало. У одного пользователя в галерее только коты, у другого — машины.
2. На H100 у нас стабильная сеть. В FL клиент может отключиться в любой момент (села батарейка, пропал Wi-Fi). Алгоритм должен быть устойчив к потере 90% воркеров во время шага.
3. Приватность: даже градиенты могут "течь". По градиенту можно восстановить куски исходных данных.

С приватностью, например, можно решать вопрос с помощью Differential Privacy (добавление шума в веса) и Secure Multi-Party Computation (сервер усредняет веса, даже не видя их в открытом виде).





Вопросы?

- **Масштабирование неизбежно:** Чтобы учить большие модели на больших данных, мы используем Data Parallelism.
- **Суть DP:** Дробим данные, копируем модель, синхронизируем (усредняем) градиенты.
- **Parameter Server:** Централизованная архитектура для хранения огромных моделей по кускам (KV-Store) и управления шагами.
- **Синхронизация:** Выбор между гарантированной сходимостью (BSP) и максимальной утилизацией железа без простоя (ASP/SSP).
- Учите плотную модель (LLM, Vision) → используйте All-Reduce (PyTorch DDP).
- Учите разреженную модель (RecSys) с огромными таблицами → используйте Parameter Server (или его современные аналоги).



Спасибо за
внимание!